Markov Decision Processes: DP + RL

In this lecture, we extend our discussion of Markov reward processes to Markov decision processes (MDPs). Examples abound:

- **A self-driving car** traveling through a city must constantly decide whether to brake, accelerate, or change lanes. Its goal is to reach its destination safely, without causing discomfort to its passengers (aggressive starting and stopping), or colliding with other cars or pedestrians.
- **A recycling robot** moves around a waste-sorting facility, deciding whether or not to discard items it encounters along the way. It aims to cover the space efficiently, while correctly identifying as many recyclables as possible.
- **A robot vacuum cleaner** moves around a house, deciding which areas to clean next. It aims to clean as much of the floor as possible, while avoiding obstacles and making sure not to exhaust its battery.
- **A student studying for an upcoming exam** and aiming to do well must decide which topics to study next, taking into account their current understanding and their remaining study time.
- **A smart thermostat** decides whether to turn the heat on or off, or to adjust its intensity based on the current indoor temperature. Its goal is to maintain a comfortable temperature while minimizing energy consumption.

For MDPs, we pose and solve the control problem: i.e., that of computing an optimal policy. Specifically, we describe value iteration, a dynamic programming algorithm that is used to compute an optimal policy in an MDP when the state space is small and dynamics are known, and then we describe two model-free reinforcement learning algorithms for control, namely Q-learning and SARSA.

1 Markov Decision Processes

An agent operating in a non-deterministic environment transitions from state to state, in general making decisions and obtaining rewards along the way, as follows: at time t,

1. state is s_t
2. choose action a_t
3. receive reward r_t
4. transition to state s_{t+1} with probability $P[s_{t+1}|s_t, a_t]$

Markov decision processes (MDPs) model such agent-environment interactions.

Formally, a (discrete-time) **Markov decision process** is a tuple $\langle S, A, R, P, \mu, \gamma \rangle$, where time is discrete: i.e., $t \in T = \{1, 2, \ldots\}$, and

[1]These notes were compiled in conjunction with Professor Eric Ewing.

- S is a finite set of states ($s \in S$)
- A is a finite set of actions ($a \in A$)
- $R : S \times A \to \mathbb{R}$ is a reward function
- $P : S \times A \to \Delta(S)$ is a probability transition function (or matrix)
  $\Delta(S)$ is the set of probability distributions over S
- μ is the initial state distribution
- γ is the discount factor

Note that we model rewards as a function of both states and actions. Doing so strictly generalizes modeling rewards as a function of states alone, and captures the potential impact of different actions that transition from the same state s to the same state s'. For example, a self-driving car that accelerates and then brakes to move from one state to another may waste energy, compared to instead moving smoothly among states.

2 State Values

A **policy** is a map from states to actions, i.e., $\pi : S \to A$, or to distributions over actions, i.e., $\pi : S \to \Delta(A)$. By Bellman's theorem, the state value $V^\pi(s)$ at state s under policy π is the sum of the immediate reward obtained in state s and the discounted sum of the rewards obtained by following policy π thereafter:

$$V^\pi(s) = R(s, \pi(s)) + \gamma \mathbb{E}_{s'}[V^\pi(s')] \tag{1}$$

N.B. We overload notation here, interpreting $R(s, \pi(s)) = \mathbb{E}_{a \sim \pi(s)}[R(s, a)]$, when π is not deterministic.

Policy π dominates policy $\hat{\pi}$ (notation $\pi \gg \hat{\pi}$) iff $V^\pi(s) \geq V^{\hat{\pi}}(s)$, for all states $s \in S$. We seek an optimal policy: i.e., π^* *s.t.* $\pi^* \gg \pi$, for all policies π. That such a policy exists is not at all obvious. Indeed, Bellman's celebrated optimality equations (Equation 2) and their accompanying dynamic program establish both the existence of an optimal value function and a corresponding optimal policy.

$$V(s) = \max_a \{R(s, a) + \gamma \mathbb{E}_{s'}[V(s')]\} \tag{2}$$

Assuming $\gamma < 1$, the optimal value function V^* is the unique solution to this system of $|S|$ equations with $|S|$ unknowns. The optimal policy π^* maps state s into an optimal action, as follows:

$$\pi^*(s) \in \arg\max_a \{R(s, a) + \gamma \mathbb{E}_{s'}[V^*(s')]\} \tag{3}$$

3 Action Values

The action value $Q^\pi(s, a)$ associated with state s and action a is defined as the sum of the immediate reward obtained by taking action a in state s and the discounted sum of the rewards obtained by following policy π thereafter:

$$Q^\pi(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[V^\pi(s')] \tag{4}$$

Restating Bellman's optimality equations in terms of action values yields:

$$Q(s, a) = R(s, a) + \gamma \mathbb{E}_{s'}[V(s')] \tag{5}$$

$$V(s) = \max_a Q(s, a) \tag{6}$$

Simplifying,

$$Q(s,a) = R(s,a) + \gamma \mathbb{E}_{s'}[\max_{a'} Q(s',a')] \tag{7}$$

As above, assuming $\gamma < 1$, the optimal action-value function Q^* is the unique solution to this system of $|S \times A|$ equations with $|S \times A|$ unknowns.

Given Q^*, the optimal policy π^*, which maps all states s into an optimal action, can be greedily extracted as follows:

$$\pi^*(s) \in \arg\max_a Q^*(s,a) \tag{8}$$

While the optimal action-value function Q^* is unique, the optimal policy π^* need not be unique.

4 Dynamic Programming: Value Iteration

The value iteration algorithm, which is based on Equation 2, updates as follows:

$$V(s) \leftarrow \max_a \{R(s,a) + \gamma \sum_{s'} P[s'|s,a] V(s')\} \tag{9}$$

Equivalently, value iteration can be described in terms of Equations 5 and 6:

$$Q(s,a) \leftarrow R(s,a) + \gamma \sum_{s'} P[s'|s,a] V(s') \tag{10}$$

$$V(s) \leftarrow \max_a Q(s,a) \tag{11}$$

The algorithm, which is depicted in Table 1, first computes the value of each state for all actions, and then sets each state's value to be the greatest value achieved among all courses of action. The actions that yield the optimal state values can then be greedily extracted as the optimal policy, as in Equation 8.

VALUE_ITERATION(MDP, γ, ϵ)	
Inputs	discount factor γ
	convergence test ϵ
Output	optimal state-value function V^*
Initialize	$V = 0$ and $V' = \infty$

while $\max_s |V(s) - V'(s)| > \epsilon$ **do**

1. $V' = V$
2. for all $s \in S$
   (a) for all $a \in A$
      i. $Q(s,a) \leftarrow R(s,a) + \gamma \sum_{s'} P[s'|s,a]\, V(s')$
   (b) $V(s) \leftarrow \max_a Q(s,a)$

return V

Table 1: Value Iteration *à la* Gauss-Seidel.

5 Reinforcement Learning: Q-Learning and SARSA

We now turn our attention to algorithms that learn action-value functions without knowledge of P or R, from which we can extract an optimal policy. We present two learning algorithms: Q-learning and SARSA.

A reinforcement learning agent learns from experience. As such, it must act, either in its environment or in a simulation of its environment, in order to gather said experience. The policy that directs the agent's actions is called its **behavior policy**. The policy whose action values the agent is trying to learn is called the **target policy**. When the behavior and target policies coincide, the algorithm is called **on-policy**; and when they don't, it is called **off-policy**.

Reinforcement learning algorithms learn in **episodes**. Each episode is a simulation, whereby the agent gathers experience in the form of a **trajectory**, i.e., $s_1, a_1, r_1, s_2, a_2, r_2, \ldots$. Each trajectory begins at a start state drawn from the initial distribution μ, and then transitions according to the agent's behavior policy and the MDP's stochastic transitions.

Exploration vs. Exploitation Unlike in dynamic programming, in the reinforcement learning framework it is not assumed that the probabilistic nature of the environment is known. Nor is it assumed that the reward structure is known. Instead, reinforcement learning agents wander through their environments learning about rewards only at the states they visit and only for the actions they employ.

Naturally, such agents would aim to reinforce, that is "become more and more likely to employ," those actions that are found to be the most rewarding. With this objective in mind, reinforcement learning agents are susceptible to the trade-offs between exploration and exploitation, as in simulated annealing, while learning action values. By *exploiting* actions that have proven successful in the past, it is possible to perform well; but by *exploring* alternative actions, it is possible to perform even better.

One popular method of exploration is ϵ-greedy: if π is a greedy policy with respect to the current Q values and s is the current state, with probability $1-\epsilon$, exploit, i.e., take action $\pi(s)$, but with probability ϵ, explore, i.e., choose an action uniformly at random. Typically, ϵ is decayed over time (e.g., $\epsilon \sim 1/t$), once again, as in simulated annealing. This technique, however, explores seemingly optimal and sub-optimal actions with equal probability. An alternative is to use the *softmax* action selection method, which relies on the Boltzmann distribution. Specifically, given state s_t, action a is selected with the following probability:

$$\frac{e^{Q(s_t,a)/T}}{\sum_{a'} e^{Q(s_t,a')/T}}$$

where the temperature parameter T gradually decreases. All actions are nearly equiprobable at initial higher temperatures; in contrast, lower temperatures extol the virtues of some actions but belittle others.

Q-Learning Whereas TD-learning is an application of Bellman's theorem for V, Q-learning is based on Bellman's optimality equations for Q:

$$Q^*(s,a) = R(s,a) + \gamma \mathbb{E}[\max_{a'} Q^*(s',a')] \tag{12}$$

After taking action a in state s, receiving reward r, and transitioning to state s', Q-learning updates as:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{13}$$

Q-learning is off-policy, because its target policy is a greedy one, while its behavior policy is exploratory. This separation is what makes Q-learning appealing: it learns about the values associated with optimal policies while behaving otherwise. Indeed, Q-learning learns Q^* under *any* behavior policy that visits all state-action pairs infinitely often, assuming α decays appropriately.

Q_LEARNING(Environment, γ)	
Inputs	discount factor γ, exploration policy, length of trajectories L, number of episodes N
Output	(hopefully, an optimal) action-value function Q^*
Initialize	$Q = 0$, α according to a decay schedule

`repeat`

1. initialize $s \sim \mu$ and a according to the exploration policy
2. `repeat`
   (a) take action a
   (b) observe reward r, next state s'
   (c) $Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$
   (d) choose action a' according to the exploration policy
   (e) $s = s'$, $a = a'$
   (f) decay α according to schedule

   `for` L `steps`

`for` N `episodes`

Table 2: Q-Learning: Off-policy reinforcement learning.

SARSA SARSA's update is inspired by the following variant of Bellman's optimality equations:

$$Q^*(s,a) = R(s,a) + \gamma \mathbb{E}[Q^*(s', \pi^*(s'))] \tag{14}$$

Note that Equation 14 is a simple rewrite of Equation 12, since π^* is a greedy policy with respect to Q^*.

To follow this recipe SARSA, learns from its namesake, namely tuples (s, a, r, s', a'). After taking action a in state s, receiving reward r, and transitioning to state s', SARSA updates as:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)] \tag{15}$$

Because a' is the agent's actual action, the target of this update is the value of following the ϵ-greedy policy. SARSA's target policy thus coincides with its behavior policy: i.e., SARSA is on-policy.

If ϵ is held fixed, the best SARSA can do is learn an optimal ϵ-greedy policy; but if $\epsilon \to 0$, so that the ϵ-greedy policy approaches π, then SARSA learns action values that approach Q^*.

SARSA(Environment, γ)

Inputs	discount factor γ, length of trajectories L, number of episodes N, initial policy π
Output	(hopefully, an optimal) action-value function Q^*
Initialize	$Q = 0$, α according to a decay schedule, ϵ according to a decay schedule

`repeat`

1. draw $s \sim \mu$
2. choose random action a, with probability ϵ
   choose action $a = \pi(s)$, with probability $1 - \epsilon$
3. `repeat`
   (a) take action a
   (b) observe reward r, next state s'
   (c) choose random action a', with probability ϵ
   choose action $a' = \pi(s')$, with probability $1 - \epsilon$
   (d) $Q(s, a) = Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
   (e) $\pi(s) \in \arg\max_{a''} Q(s, a'')$
   (f) $s = s'$, $a = a'$
   (g) decay α and ϵ according to schedule

   `for` L `steps`

`for` N `episodes`

Table 3: SARSA: On-policy Reinforcement Learning.